

Nome:**Num:****Duração Total Exame (T + P) :** 2h:00m + 30m tolerância

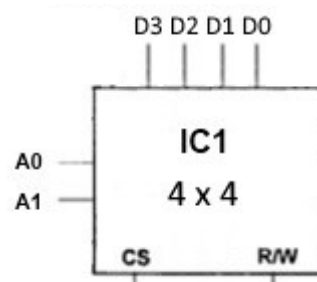
11 de Julho de 2022

Parte Teórica

- 1** Considere uma máquina com uma cache com mapeamento direto, blocos de 1 Byte e com 7 bits de *tag*. Esta máquina tem uma memória RAM com 2KB de capacidade.
- a) Qual a função da *tag*? Justifique. **(0,5 Val.)**
 - b) Qual a função dos *valid bits*? Justifique. **(0,5 Val.)**
 - c) Qual a capacidade total desta cache, contando com os bits da *tag* mais os *valid bits*? **(2 Val.)**

- 2** Considere o circuito integrado de memória RAM da figura, onde A_1 , A_0 representam linhas de endereço D_3 , D_2 , D_1 , D_0 representam linhas de dados, R/W representa a linha de leitura/escrita e CS a linha de *Chip Select*.

Pretende-se dimensionar uma memória capaz de armazenar 15 bytes. Faça um esboço, associando múltiplos circuitos integrados do tipo IC1, de forma a obter a memória RAM pretendida. Deverá ser indicada explicitamente a linha de CS (*Chip Selection*) da memória resultante.

**(2 Val)**

- 3** As tecnologias de memória utilizadas em dispositivos de armazenamento secundário apresentam uma evolução considerável das suas características mais relevantes desde o aparecimento dos primeiros computadores até ao dia de hoje. O armazenamento em cartão e papel perfurado, passando pela gravação magnética em fita e discos, bem como o armazenamento ótico são as tecnologias mais antigas. Mais recentemente o armazenamento de estado sólido (Memória **Flash**) adquiriu uma importância crescente. Faça uma descrição detalhada dos seguintes aspectos deste tipo de memória: **(2 Val)**
- a) O processo físico que permite representar informação binária através da memória Flash.
 - b) Representação da informação na variante MLC “Multi Level Cell”
 - c) Técnica “Die-Stacking”
 - d) Tecnologia “Wear Leveling”

Parte Prática

1. Complete os blocos de código abaixo em Assembly 8086, de forma a realizar as ações indicadas:
- a. Declare dois vetores: o primeiro com 5 números inteiros entre 0 e 255 e os caracteres A e \$; o segundo com 6 valores inteiros diferentes em que o primeiro é 0 e o último é 60000. **(0,5 Val.)**

```
dados segment                para 'data'
```

```
dados ends
```

- b. Adicione o valor 1 a todos os elementos dos dois vetores declarados abaixo. **(0,6 Val.)**

```
.8086
.model small
.stack 2048
dados segment para 'data'
    Vetor1 db 1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,'$'
    Vetor2 dw 6,7,8,9,10,11,12,20,30,40,50,60,70,80,2,3,5,8,'$'
```

```
dados ends
```

```
codigo segment para 'code'
main proc
    assume cs:codigo, ds:dados
    mov ax,dados
    mov ds,ax
```

```

    mov ah,4Ch
    int 21h
main endp
codigo ends
end main
```

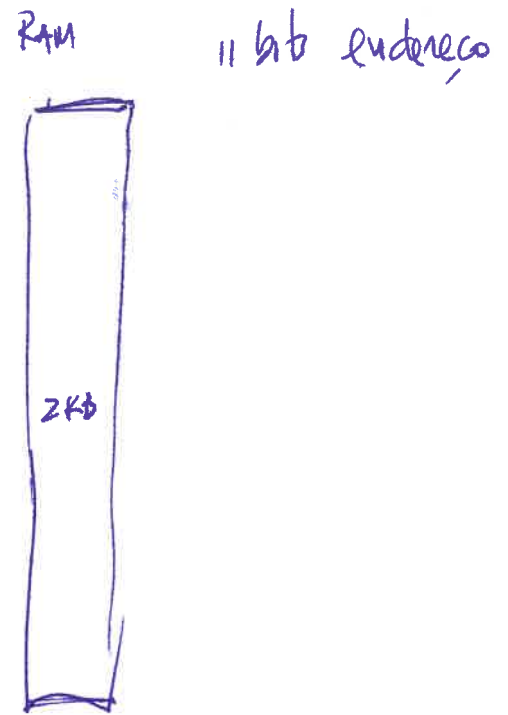
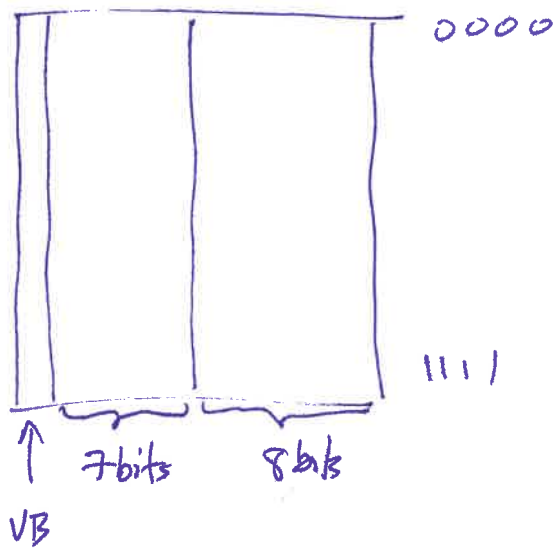
- c. Sabendo que foram declaradas duas variáveis: var1 (word) e var2 (byte), respetivamente com os valores 350 e 2, multiplique as mesmas. **(0,6 Val.)**

```
...  
codigo      segment para 'code'  
    main    proc  
        assume cs:codigo, ds:dados  
        mov  ax,dados  
        mov  ds,ax
```

```
        mov  ah,4Ch  
        int  21h  
main    endp  
codigo  ends  
end     main
```


[illegible]

①



$$\begin{aligned} \text{RAM} &= 2 \text{ KB} = 2 \cdot 1024 \cdot \text{B} \\ &= 2 \cdot 2^{10} = 2^{11} \end{aligned}$$

$$n=11 = 7 + \text{endereço cache}$$

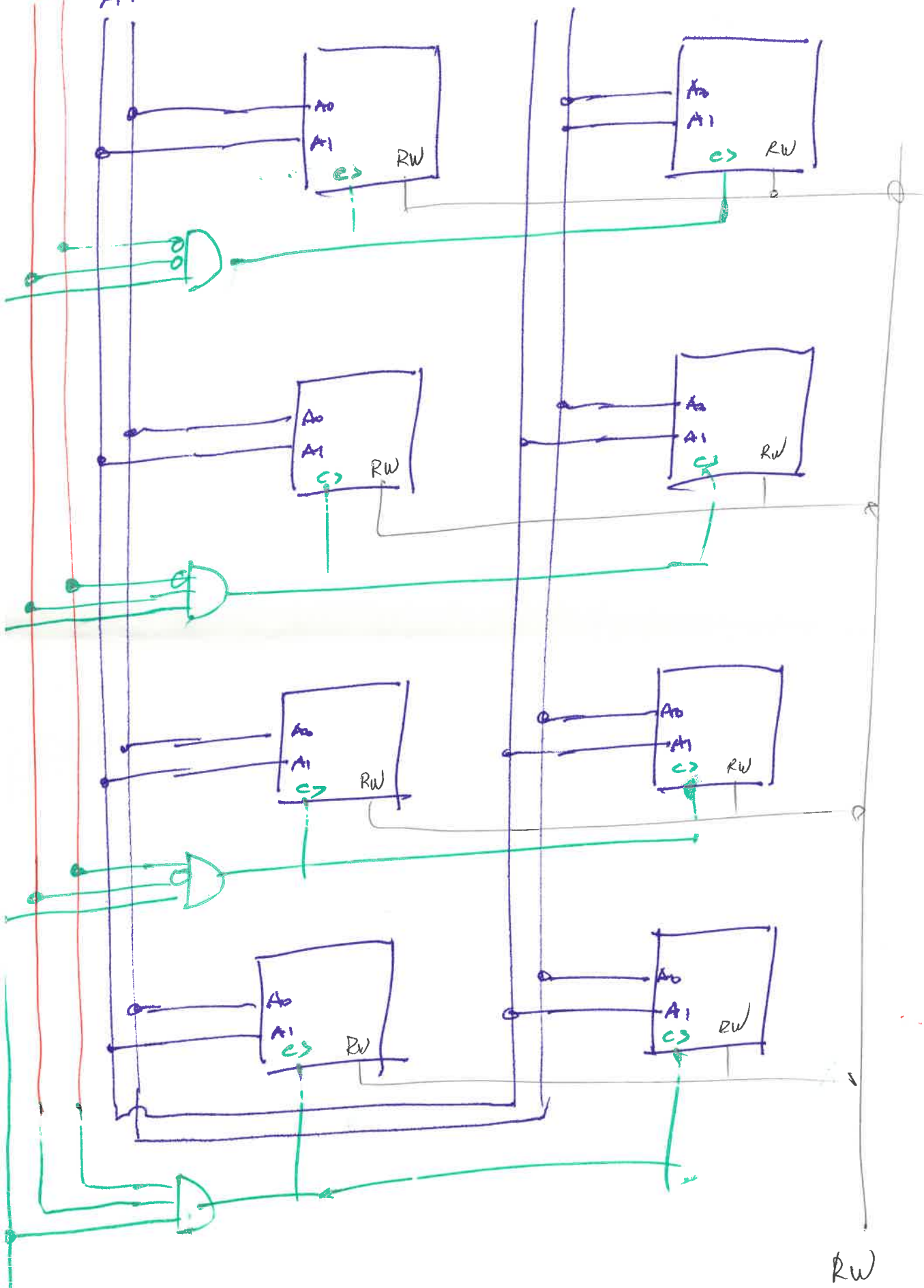
$$n = 11 - 7 = 4 \text{ bits}$$

~~cap. cache = 16 × 16 = 256 bits = 32 bytes~~

e) Cap. cache (TOTAL) = 16 × 16 = 256 bits = 32 bytes

2

A₃ A₂ A₁ A₀



1. Complete os blocos de código abaixo em Assembly 8086, de forma a realizar as ações indicadas:
 - a. Declare dois vetores: o primeiro com 5 números inteiros entre 0 e 255 e os caracteres A e \$; o segundo com 6 valores inteiros diferentes em que o primeiro é 0 e o último é 60000.

```
dados segment          para 'data'

    Squares byte 0, 1, 4, 9, 16, 'A', '$'
    Big dd 0, 10, 400, 900, 1600, 60000

dados ends
```

- b. Adicione o valor 1 a todos os elementos dos dois vetores declarados abaixo.

```
.8086
.model small
.stack 2048
dados segment para 'data'
    Vetor1 db 1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,'$'
    Vetor2 dw 6,7,8,9,10,11,12,20,30,40,50,60,70,80,2,3,5,8,'$'
dados ends
codigo segment para 'code'
    main proc
        assume cs:codigo, ds:dados
        mov ax,dados
        mov ds,ax

        xor bx, bx
        xor si, si
    ciclo1:
        mov al, Vetor1[bx]
        cmp al, '$'
        je ciclo2
        inc al
        mov Vetor1[bx], al
        inc bx
        jmp ciclo1
    ciclo2:
        mov ax, Vetor2[si]
        cmp al, '$'
        je sai
        inc ax
        mov Vetor2[si], ax
        inc si
        inc si
        jmp ciclo2
    sai:
        mov ah,4Ch
int 21h
main endp
codigo ends
end main
```


- c. Sabendo que foram declaradas duas variáveis: var1 (word) e var2 (byte), respectivamente com os valores 350 e 2, multiplique as mesmas.

```
...
codigo      segment para 'code'
    main proc
        assume cs:codigo, ds:dados
        mov ax,dados
        mov ds,ax

        mov ax, var1

        mov bl, var2
xor al,al → xor bh, bh
        mul bx

        mov ah,4Ch
        int 21h
    main endp
codigo      ends
end main
```

- d. Sabendo que foi declarado um vetor de caracteres que termina com '0', altere o mesmo, de modo a que todos os caracteres minúsculos se transformem no carácter seguinte do abecedário (por exemplo: 'a' passa a 'b', 'b' passa a 'c'). No caso do 'z' este deve passar a 'a'.

```
...
codigo      segment para 'code'
main proc
    assume cs:codigo, ds:dados
    mov ax,dados
    mov ds,ax

    xor si,si
cic: mov al,string[si]
    cmp al,'0'
    je SAI
    cmp al, 'a'
    jb Proximo
    cmp al, 'z' - 1
    ja  TestaZ
    inc al
    mov string[si], al
    jmp Proximo
TestaZ:
    cmp al, 'z'
    jne Proximo
    mov string[si], 'a'
Proximo:
    inc si
    jmp cic
SAI: mov ah,4Ch
    int 21h
main endp
codigo ends
end main
```

```
;#####
; AH = LINHA INICAL
; AL = COLUNA INICIAL
; SI, = Endereço da palavra a encontrar (termina com o caracter '$')
; O procedimento deverá devolver 1 no registo AH se encontrou a palavra ou 0 caso contrário.
```

```
;##### Critérios: #####
; push/pops = 2.5%
; procs e endp = 5%
; calcular posição inicial = 20%
; comparar 1º carater = 5%
; Ciclo percorrer palavra ate ao fim = 15%
; Verificar cada Direcção = 50%
; ret = 2,5%
```

VERIFICA_DIAGONAL Proc

```
    pushf
    push bx
    push cx
    push dx
    push di

    mov ch,ah
    mov bl,2
    mul bl
    mov bx,ax
    mov al,160
    mul ch
    add bx,ax           ;Posição inicial em bx

    mov al,[si]         ;Compara 1º carater
    inc si
    cmp al,es:[bx]
    jne notFound        ;1º Carater diferente

    mov di,si           ;Preserva valores
    mov dx,bx
    mov cx,4            ;cx - direcção
```

```
nextCar:  mov al,[si]
          cmp al,'$'
          je found
          inc si
          cmp cl,4
          je topLeft
          cmp cl,3
          je topRight
          cmp cl,2
          je bottomLeft
```

```
bottomRight: add bx,162
             jmp check
```

```
topLeft:     sub bx,158
             jmp check
```

```
topRight:    sub bx,162
             jmp check
```

```
bottomLeft:  add bx,158
```

```
check:       cmp al,es:[bx]
             je nextCar
```

```
newDirection:
             mov si,di           ;ResetPosicao
             mov bx,dx
             loop nextCar        ;Testa nova direcção
```

```
notFound:    mov ah,0
             jmp fim

found:       mov ah,1
fim:
            pop di
            pop dx
            pop cx
            pop bx
            popf
            ret
VERIFICA_DIAGONAL endp
```